Dokumentacja inżynierii wymagań

Projekt: aplikacja do nauki języków obcych poprzez rozmowę z AI, z korekcją błędów

Członkowie zespołu: Łukasz Skrzypek, Maciej Antosz, Arek Kępa

Zamawiający: LanguageMate Sp. z o.o. (Paweł Lewandowski). Zamówienie dotyczy aplikacji z asystentem AI, który prowadzi z użytkownikiem naturalne dialogi w języku obcym, dostosowane do jego poziomu, poprawia błędy, odpowiada i zadaje pytania ćwiczące rozmowę. Raport zawiera macierz kompetencji, pytania doprecyzowujące oraz model systemu.

1. Macierz kompetencji zespołu

Kompetencje	Łukasz Skrzypek	Maciej Antosz	Arek Kępa
Programowanie Python	Posiada	Posiada	Posiada
Bazy danych	Posiada	Posiada	Posiada
Znajomość UML	Nie posiada	Nie posiada	Posiada
Integracja modeli językowych (LLM)	Posiada	Nie posiada	Nie posiada
Zarządzanie projektem	Nie posiada	Posiada	Nie posiada
Znajomość języków obcych (do testowania aplikacji)	Angielski, hiszpański	Angielski, hiszpański	Angielski, hiszpański

2. Pytania doprecyzowujące i odpowiedzi

Informacje z zamówienia nie wystarczają do rozpoczęcia prac: nie podano języków, urządzeń, sposobu korekty błędów, wyboru poziomu ani zakresu zapisywanych postępów. Poniższe pytania to uzupełniają, a odpowiedzi ustalono w trakcie dyskusji.

Pytanie	Odpowiedź	Uwagi
Jakie języki mają być obsługiwane?	Polski, angielski, hiszpański.	Zespół testuje aplikację w językach angielskim i hiszpańskim.
Jak ma wyglądać mechanizm korygowania błędów użytkownika?	Użytkownik rozmawia z AI wyłącznie pisemnie, w czacie. Wykryte błędy trafiają na listę błędów.	Zdanie z dwoma błędami liczone jest jako jeden błąd.
Na jakie urządzenia ma być aplikacja?	Desktop / WEB.	Brak aplikacji mobilnej w zakresie projektu.
Czy aplikacja ma śledzić i	Tak. Postępy są zapisywane w koncie	Użytkownik może

Pytanie	Odpowiedź	Uwagi
zapisywać postępy użytkownika?	użytkownika w postaci statystyk.	zresetować statystyki.
Czy ma być wybór poziomu zaawansowania, czy program sam wykrywa go po wstępnej rozmowie?	Poziom wybiera użytkownik.	Automatyczne wykrywanie poziomu poza zakresem.
Jak użytkownik uzyskuje dostęp do aplikacji?	Logowanie: e-mail i hasło, z funkcją przypomnienia hasła.	
Czy aplikacja korzysta z usług zewnętrznych?	Tak, z zewnętrznego API modelu językowego (LLM).	Integrację LLM zna tylko jeden członek zespołu.
Jakie statystyki mają być prezentowane?	Procent błędów (zdania poprawne i błędne), liczba słów na minutę, liczba słów wypowiedzianych, liczba unikalnych słów.	Statystyki można zresetować.
Czy dokumentacja ma zawierać projekt interfejsu?	Tak, w postaci mocku interfejsu.	Patrz punkt 8.

3. Ustalony format danych

Użytkownicy	Rozmowy	Wiadomości	Błędy	Statystyki (raport)
id (int, unique) email (string, unique) password_hash (string) poziom (wybierany przez użytkownika)	id (int, unique) user_id (int) język (PL / EN / ES) poziom start_ts, end_ts (epoch)	id (int) rozmowa_id (int) autor (user / ai) tekst (string) timestamp (epoch)	id (int) wiadomość_id (int) zdanie (string) korekta (string) opis (string)	user_id (int) język zdania_poprawne, zdania_błędne (int) procent_błędów (float) słowa_na_minutę (float) słowa_wypowiedziane (int) słowa_unikalne (int)

Warunki kontrolne

- Zdanie zawierające jeden lub więcej błędów liczy się jako jeden błąd.
- procent_błędów = zdania_błędne / (zdania_poprawne + zdania_błędne) × 100.
- słowa_na_minutę = liczba słów użytkownika / czas trwania rozmowy w minutach (end_ts − start_ts).
- Reset statystyk zeruje liczniki użytkownika dla wybranego języka.
- Adres e-mail musi być unikalny, a hasło przechowywane wyłącznie w postaci skrótu.

Przykładowa linia z danymi wiadomości: `1730000000,12,user,Yo tiene veinte años,es` (timestamp, rozmowa_id, autor, tekst, język)

Przykładowy rekord błędu: `{"wiadomość_id": 41, "zdanie": "Yo tiene veinte años", "korekta": "Yo tengo veinte años", "opis": "Błędna odmiana czasownika tener"}`

4. Model systemu

4.1. Modelowany system za pomocą tabeli

Aktorzy	Użytkownik (osoba ucząca się języka), zewnętrzne API LLM
Opis	Celem systemu jest umożliwienie nauki języka (polski, angielski, hiszpański) poprzez pisemną rozmowę z modelem językowym w czacie. System wykrywa błędy użytkownika, zapisuje je na liście błędów i prezentuje statystyki postępów.
Dane	Dane konta (e-mail, hasło), wybrany język i poziom, wypowiedzi użytkownika (tekst), odpowiedzi i korekty z LLM, statystyki
Wyzwalacz	Zalogowany użytkownik wybiera język i poziom, a następnie rozpoczyna rozmowę
Odpowiedź	W trakcie rozmowy: odpowiedź rozmówcy AI. Po rozmowie: lista błędów z korektami oraz statystyki (procent błędów, słowa na minutę, liczba słów wypowiedzianych, liczba unikalnych słów)
Uwagi	Zdanie z dwoma błędami to jeden błąd. Poziom wybiera użytkownik. Aplikacja w wersji desktop i WEB. Użytkownik może zresetować statystyki.

4.2. Przedstawienie modelowanego systemu za pomocą diagramów UML

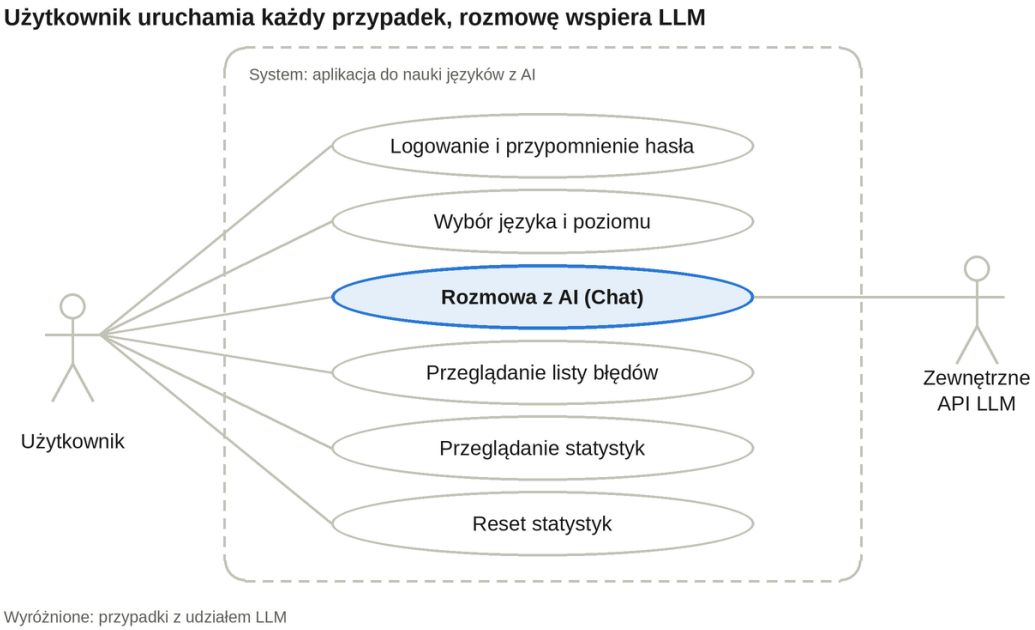
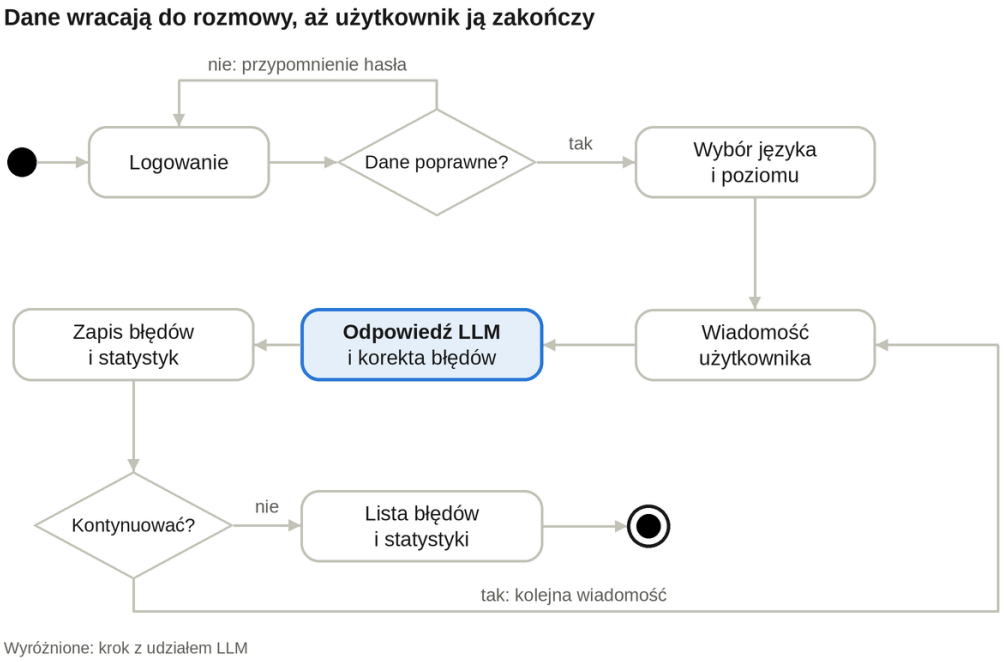


diagram przypadków użycia · 6 przypadków, 2 aktorów

Rys. 1 Diagram przypadków użycia



przepływ danych · 2 decyzje, pętla rozmowy

Rys. 2 Diagram przepływu danych

5. Diagram sekwencyjny UML

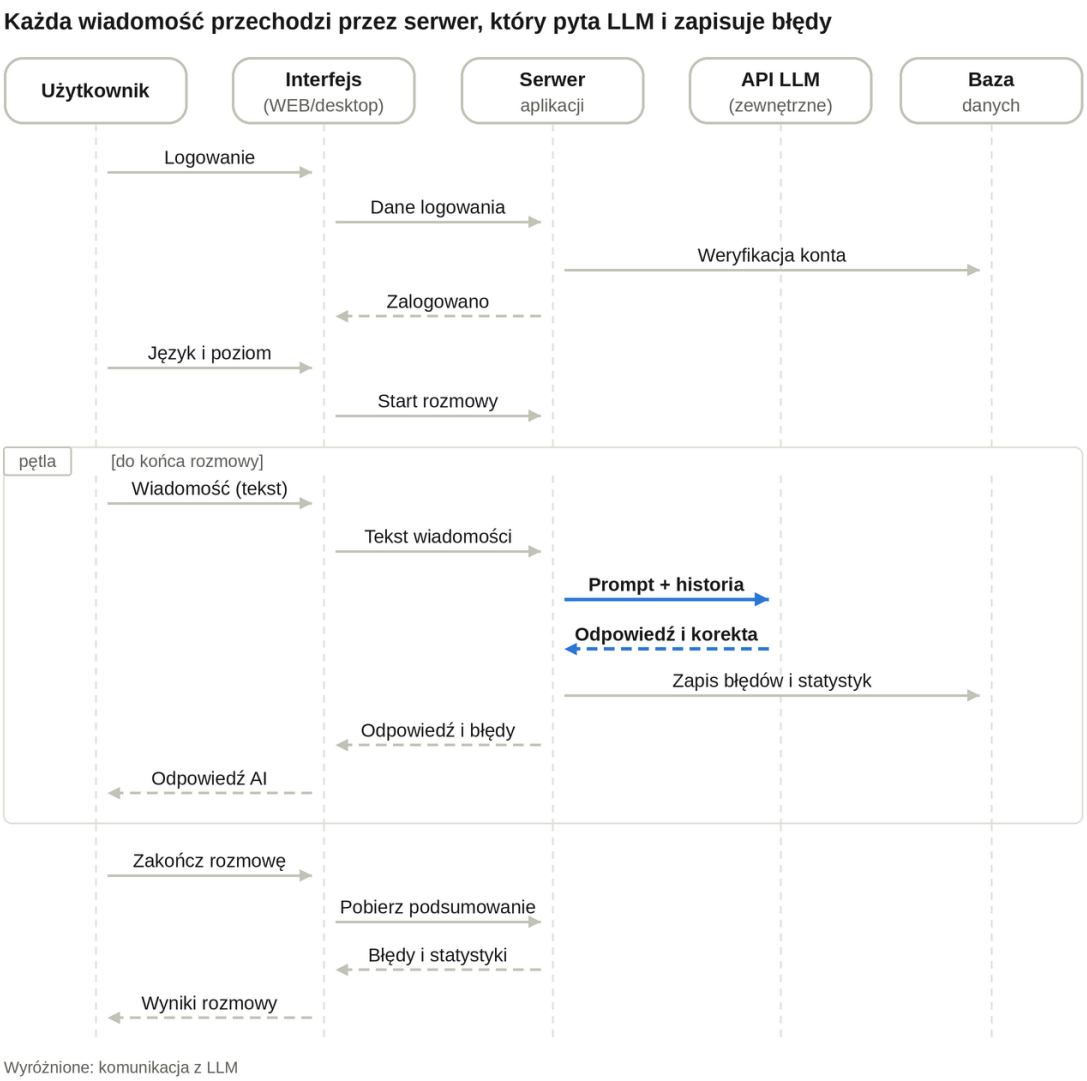
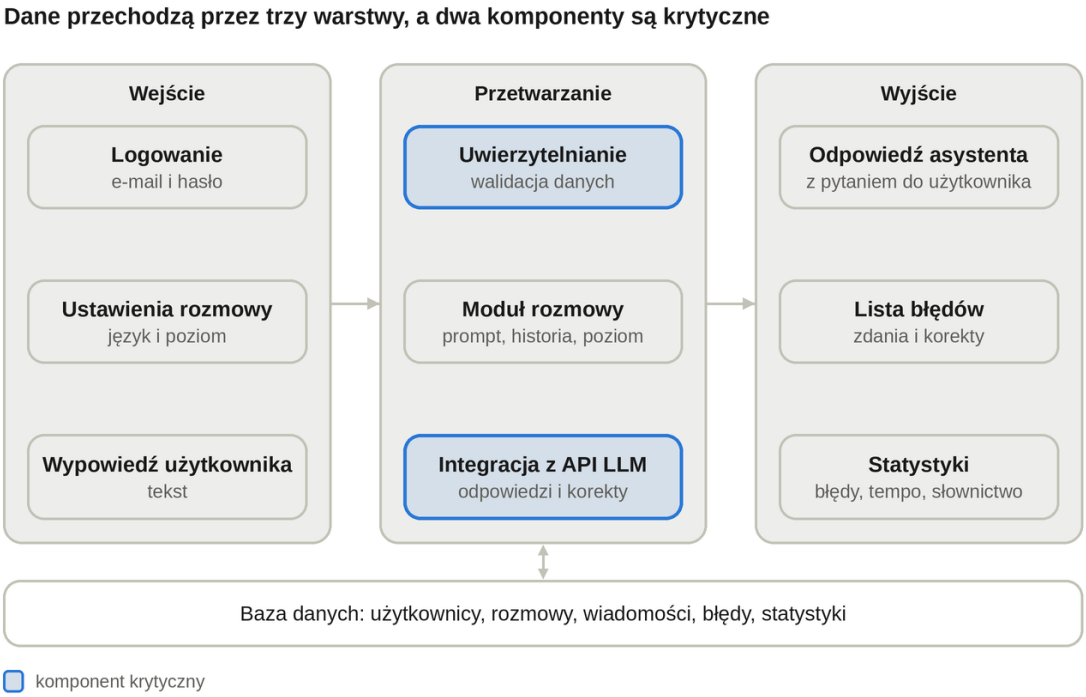


diagram sekwencyjny · 5 uczestników, pętla rozmowy

Rys. 3 Diagram sekwencyjny – rozmowa z korekcją błędów

6. Projekt architektury opracowywanego systemu



architektura · 3 warstwy, 2 komponenty krytyczne

Rys. 4 Architektura systemu

System składa się z trzech warstw: wejścia, przetwarzania i wyjścia. Zadaniem systemu jest najpierw przyjąć dane użytkownika (tekst wraz z wybranym językiem i poziomem), następnie przetworzyć je przez moduł rozmowy i model językowy, a na końcu zaprezentować odpowiedź, listę błędów oraz statystyki.

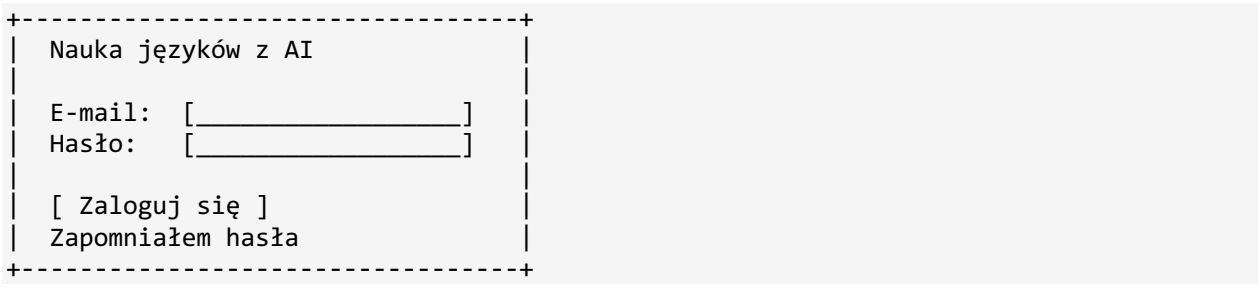
Pierwszym kluczowym i krytycznym elementem jest uwierzytelnianie oraz walidacja danych wejściowych, w tym poprawna obsługa logowania i przypominania hasła. Drugim ważnym komponentem jest integracja z zewnętrznym API LLM, odpowiedzialna za prowadzenie rozmowy i wykrywanie błędów. Jej jakość i czas odpowiedzi decydują o użyteczności aplikacji, dlatego wymaga ona starannego zaprojektowania promptów i obsługi błędów połączenia. Wszystko to stanowi podstawę do wyliczenia statystyk, które są końcową, niekrytyczną częścią systemu, ponieważ zależą od poprawnego przepływu danych przez poprzednie komponenty. Dane użytkowników, rozmów, błędów i statystyk przechowywane są w bazie danych.

7. Sugerowany język implementacji: Python

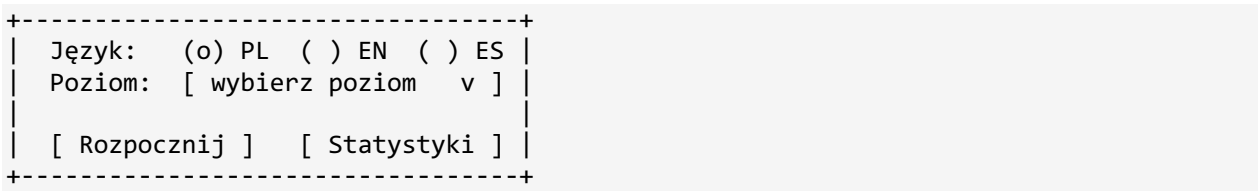
Uzasadnienie: wszyscy członkowie zespołu znają Python, co ogranicza ryzyko i koszt wdrożenia. Język ma dojrzałe biblioteki do komunikacji z zewnętrznymi API (w tym API modeli językowych), do pracy z bazami danych, w których zespół ma kompetencje. Umożliwia też szybkie prototypowanie, a frameworki webowe pozwalają wystawić tę samą logikę dla wersji WEB i desktop.

8. Mock interfejsu

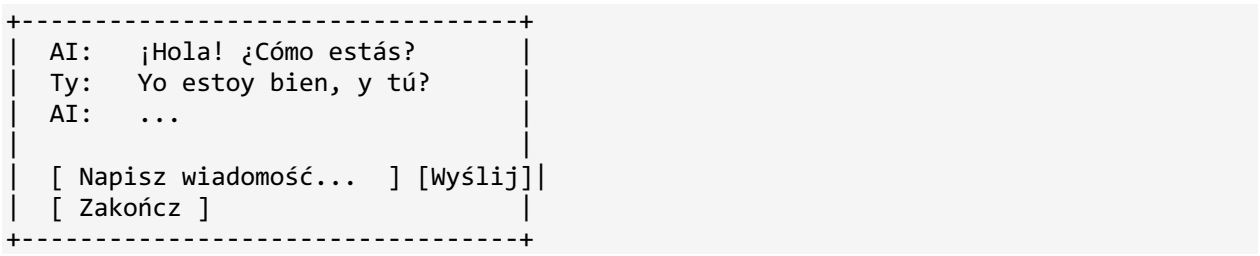
Ekran 1. Logowanie



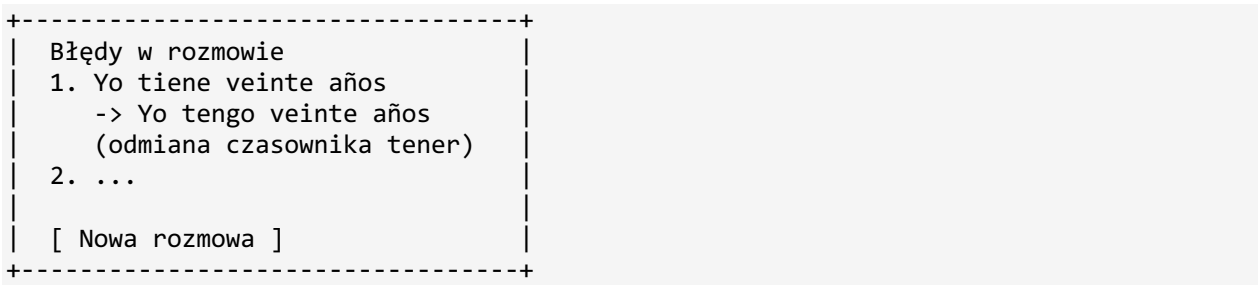
Ekran 2. Wybór języka i poziomu



Ekran 3. Rozmowa (Chat)



Ekran 4. Lista błędów po rozmowie



Ekran 5. Statystyki

